

软件工程

第3章 需求分析

3.2 用例分析(第2部分：如何基于用例描述创建顺序图)

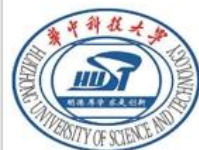
回顾：如何展开用例分析？

1.

- 用例的图形化表示——UML顺序图

2.

- 如何将用例表示为顺序图？



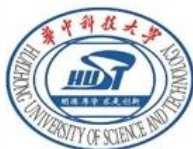
将用例表示为顺序图的基本思路

以“**边界类**”对象负责接收来自主动执行者（active actors）的消息

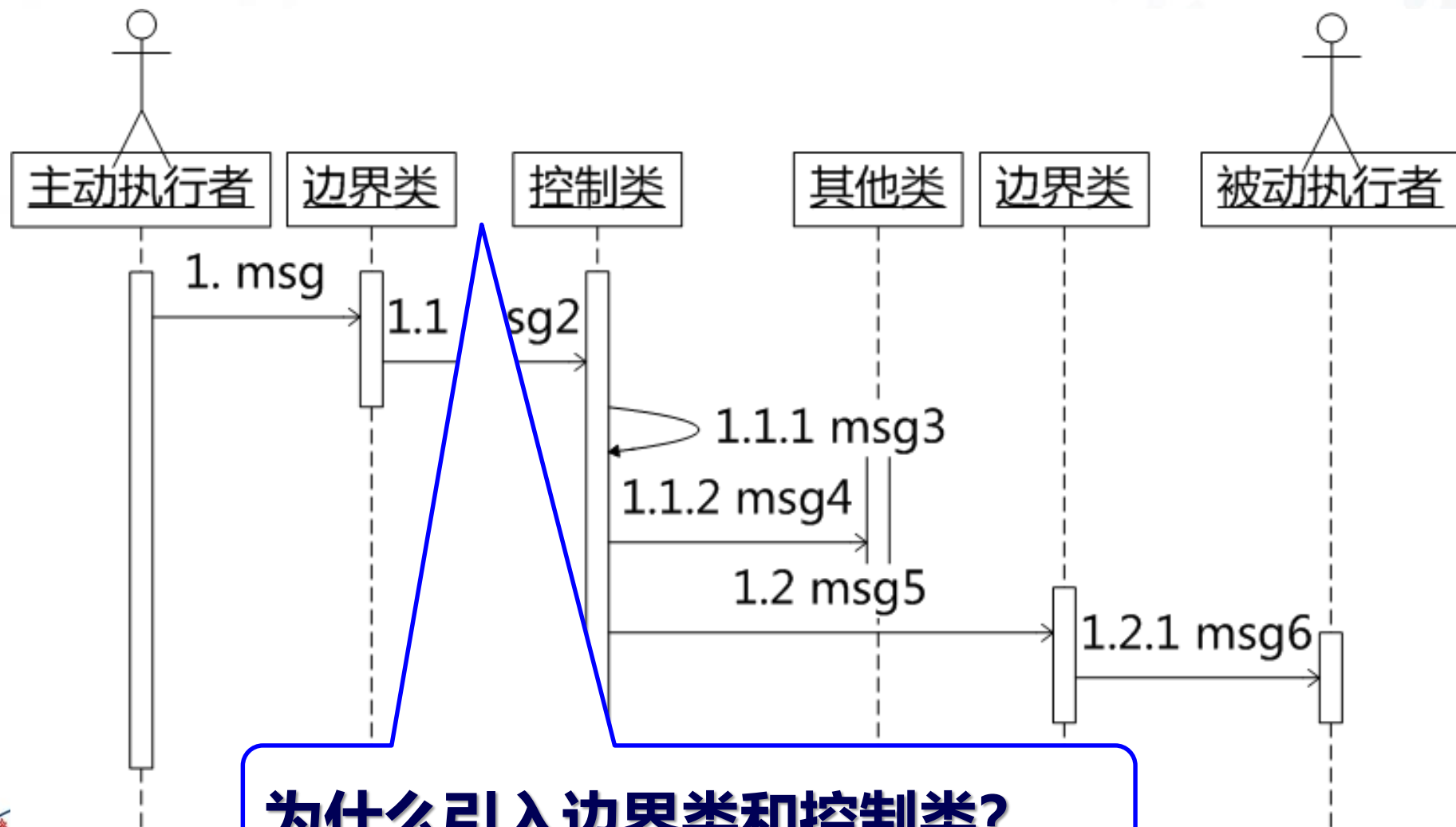
边界类对象向用例任务的主要承担者“**控制类**”对象发送命令

控制类对象独立完成命令要求的功能，或者分派子功能给其他对象

控制类对象综合处理结果，将最终结果发给负责与被动执行者交互的边界类对象



上述思路的顺序图表示



为什么引入边界类和控制类？

为什么引入边界类和控制类？

□ 面向对象软件开发方法学的基本思想：

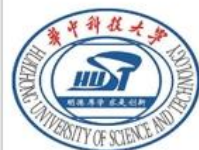
◆ 通过分属不同类的多个对象之间的协作，完成某项较复杂的任务

☑ 反对：设置一个庞大的万能对象，独立承担较复杂的任务

◆ 每个类负责特定的**职责**

☑ 边界类：负责与执行者交互，……

☑ 控制类：负责任务分派、协调，……



边界类的职责(详)

□ 界面控制：

- ◆ 输入数据的格式或内容转换；
- ◆ 输出结果的呈现；
- ◆ 软件运行过程中界面的变化与切换；等

□ 外部接口：

- ◆ 目标软件系统与外部系统或外部设备之间的信息交换和互操作



边界类的职责(续)

□ 环境隔离：

- ◆ 将目标软件系统与环境软件等进行交互的功能与特性封装于边界类，使目标软件系统的其余部分尽可能地独立于环境软件

☑ 环境软件示例：操作系统、数据库管理系统、应用服务器中间件

□ 为什么隔离？

- ◆ 提高目标软件系统的可修改性、可移植性



控制类的职责(详)

□ 作为用例任务的责任承担者，负责协调、控制其他类的对象共同完成用例规定的功能或行为

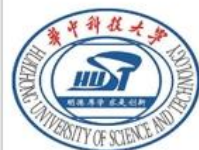
◆ 任务分解

◆ 子任务分派

◆ 协调（例如，将已完

进一步的问题：控制类的对象
如何完成任务？

参数发给这些子任务的承担者）及综合（例如，基于各子任务的
处理结果生成最终结果）



控制类如何完成任务？

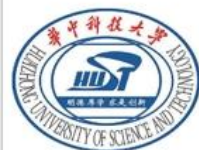
□ 如果任务非常简单：

◆ 控制类的对象独立完成

□ 如果任务比较复杂：

◆ 控制类的对象可寻求**辅助类**、**实体类**，甚至其他控制类的对象的帮助

☑ 即，分派任务给它们，综合它们的处理结果，形成最终结果

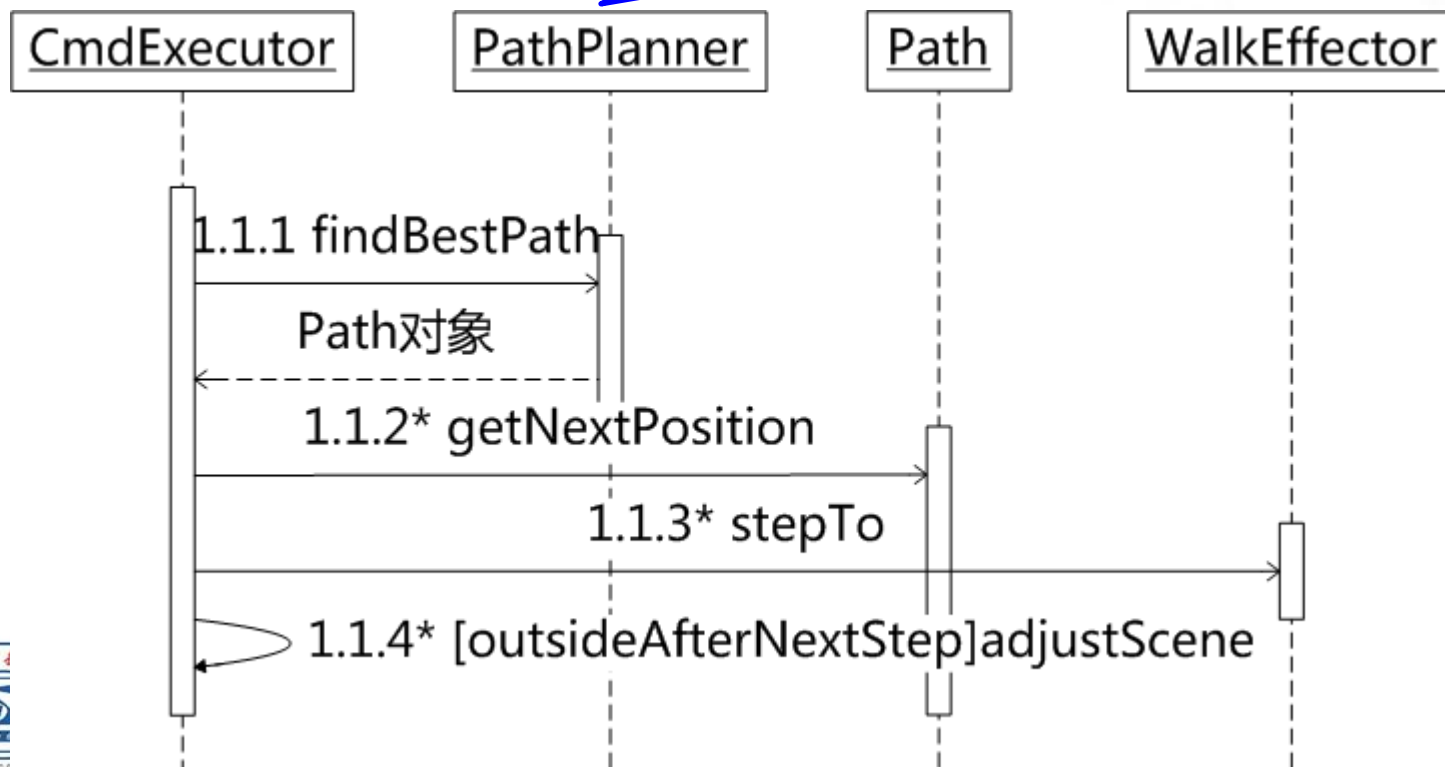


什么是辅助类?

辅助控制类完成子任务的类，例如“PathPlanner”

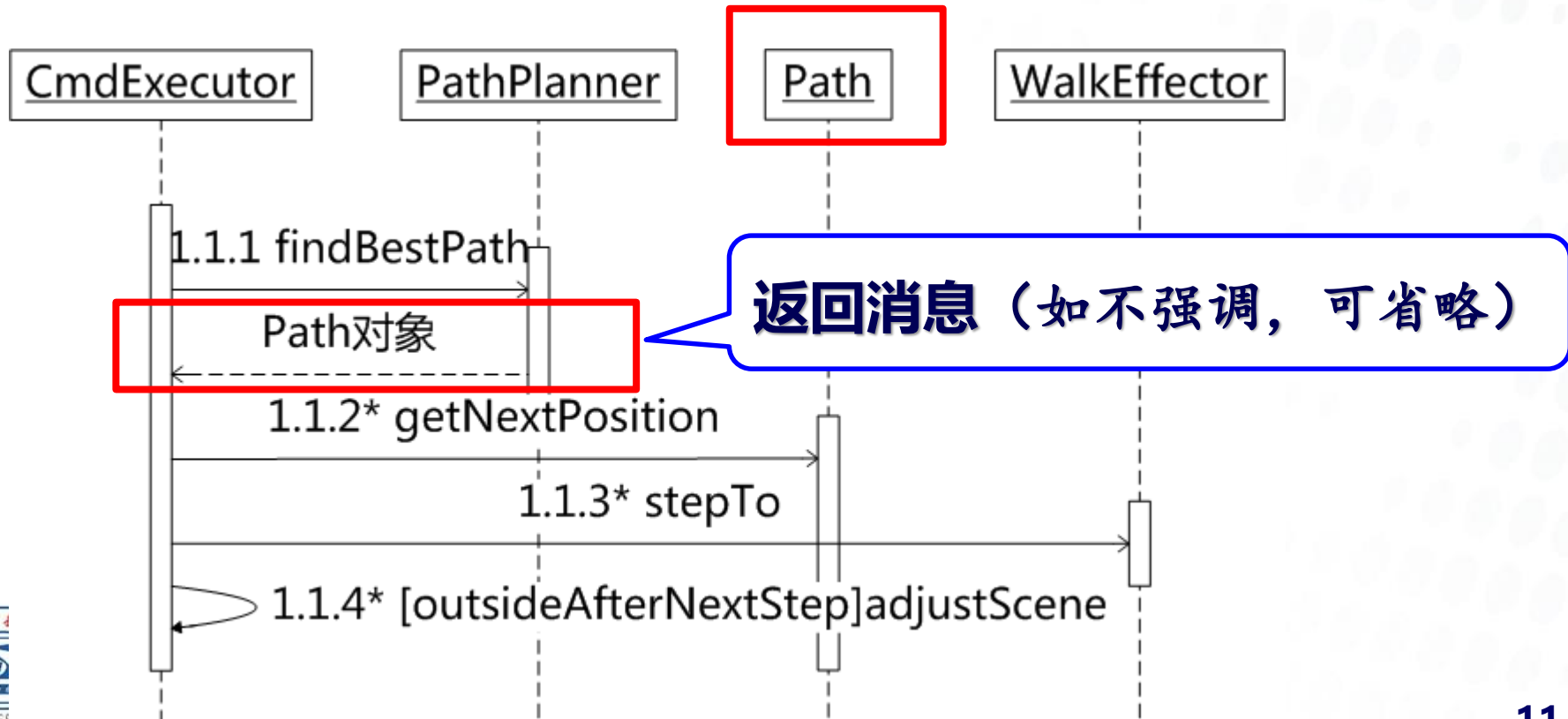
控制类的对象

辅助类的对象



什么是实体类?

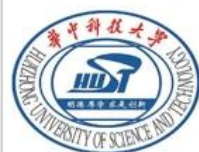
负责保存目标软件系统中具有持久意义的信息项并向其他类提供信息访问的操作，例如“Path”



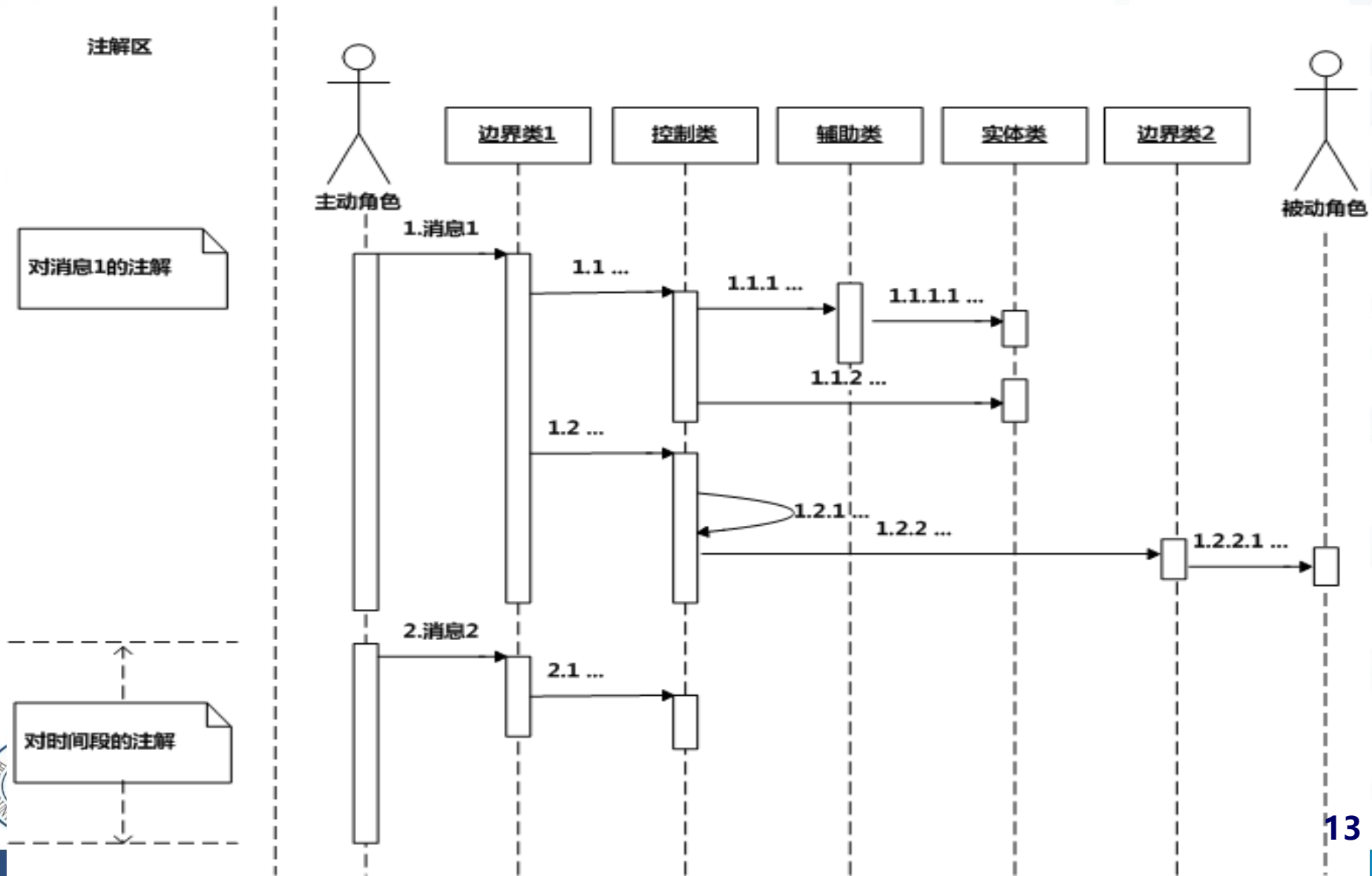
实体类的特征

□ “内敛”：

- ◆ 仅向目标软件系统的其余部分提供读、写信息项内容的操作接口
- ◆ 一般不涉及（复杂的）业务逻辑处理



小结：表示用例的顺序图的样板



如何展开用例分析？

标识分析类（边界类、控制类、实体类、辅助类）

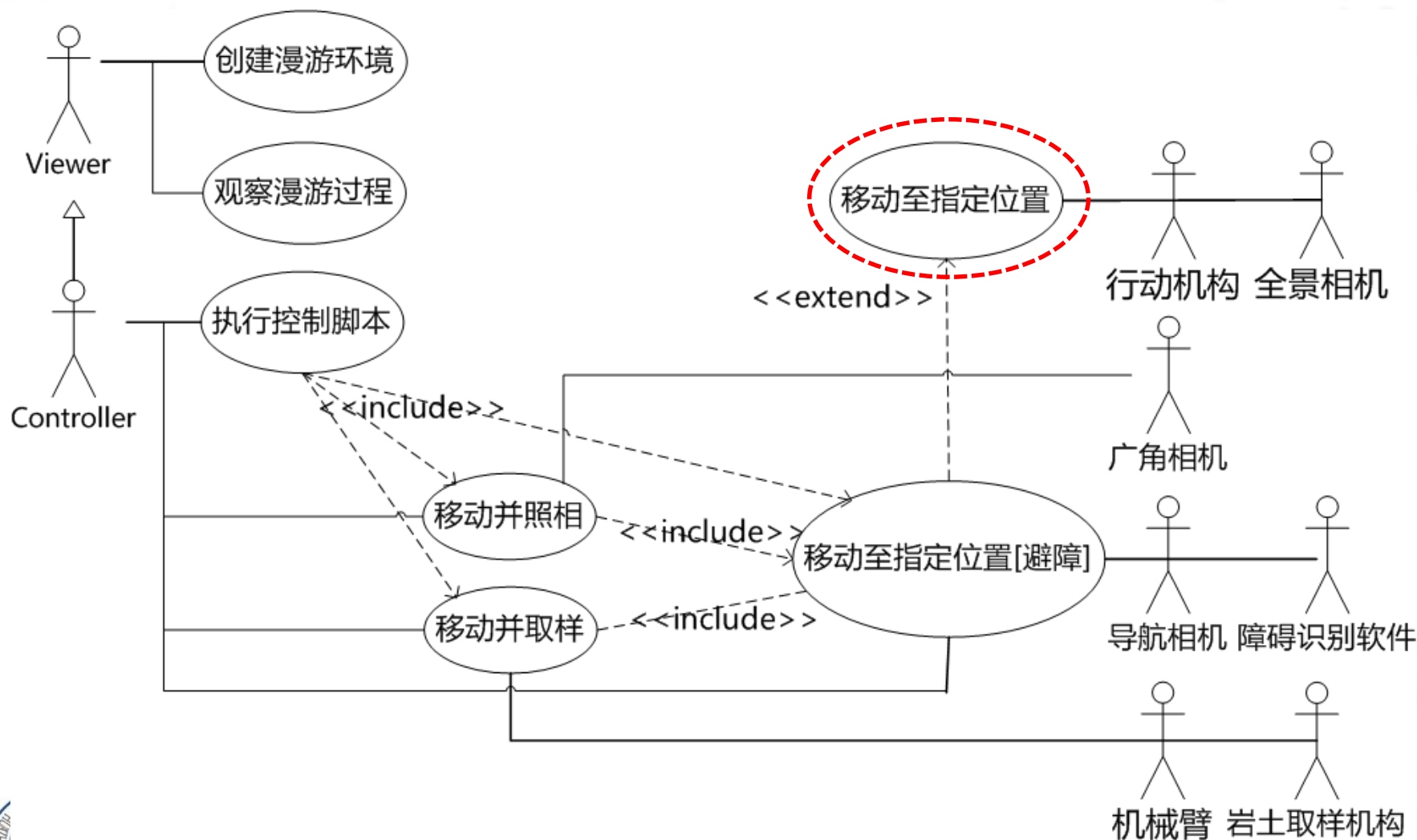
基于样板 构造用例的顺序图表示

.....

让我们再回到月球车控制仿真问题(MCS)

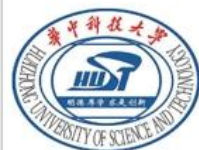


MCS的用例图



“移动至指定位置”用例的基本交互动作序列

1. 用户指定目的坐标；系统规划从当前位置到达指定位置的最佳路径；
2. 系统发出移动指令；在移动完成后，系统指示全景相机拍照留存，
 - 2.1 系统判断月球车是否已经移出当前画面，如是，移动画面以保证月球车始终位于当前画面之内；
 - 2.2 根据规划路径发出下一步移动指令，如此重复直至到达路径终点。

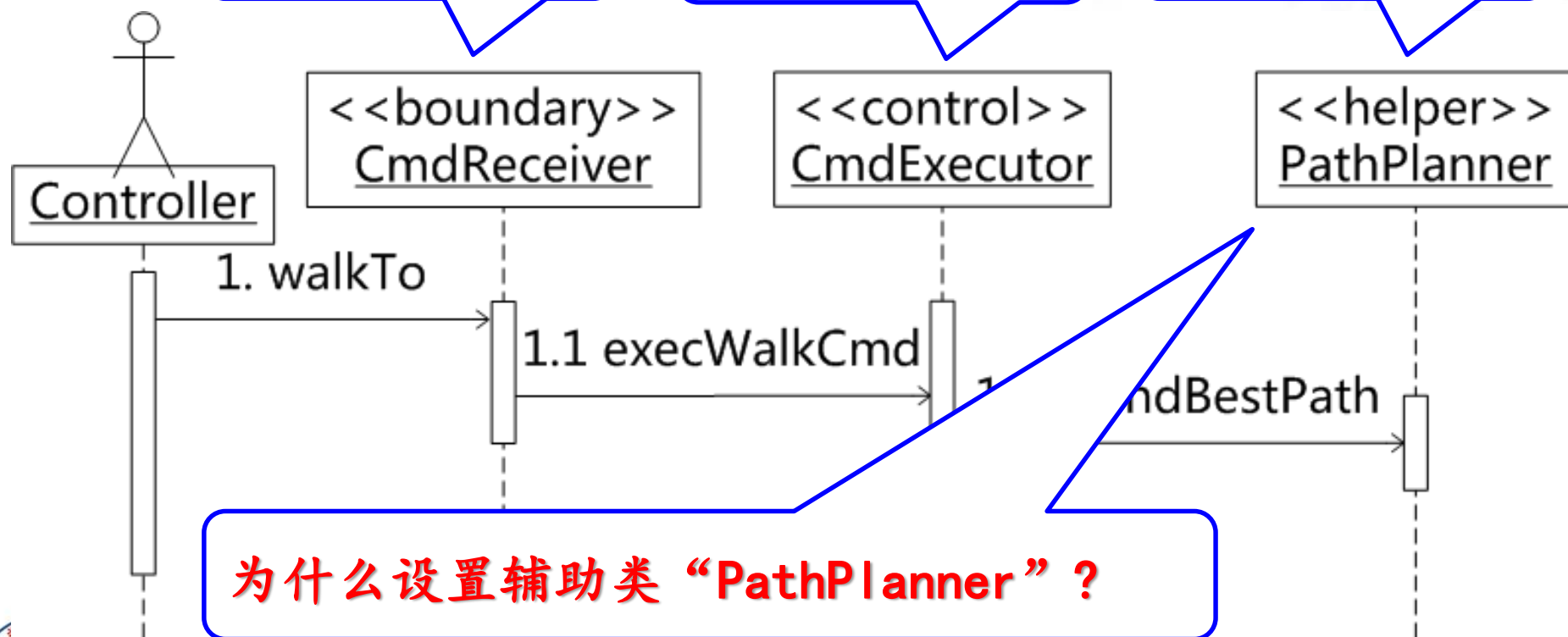


“移动”用例的顺序图表示（第1步）

边界类的对象

控制类的对象

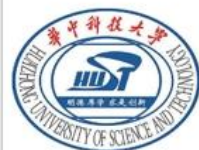
辅助类的对象



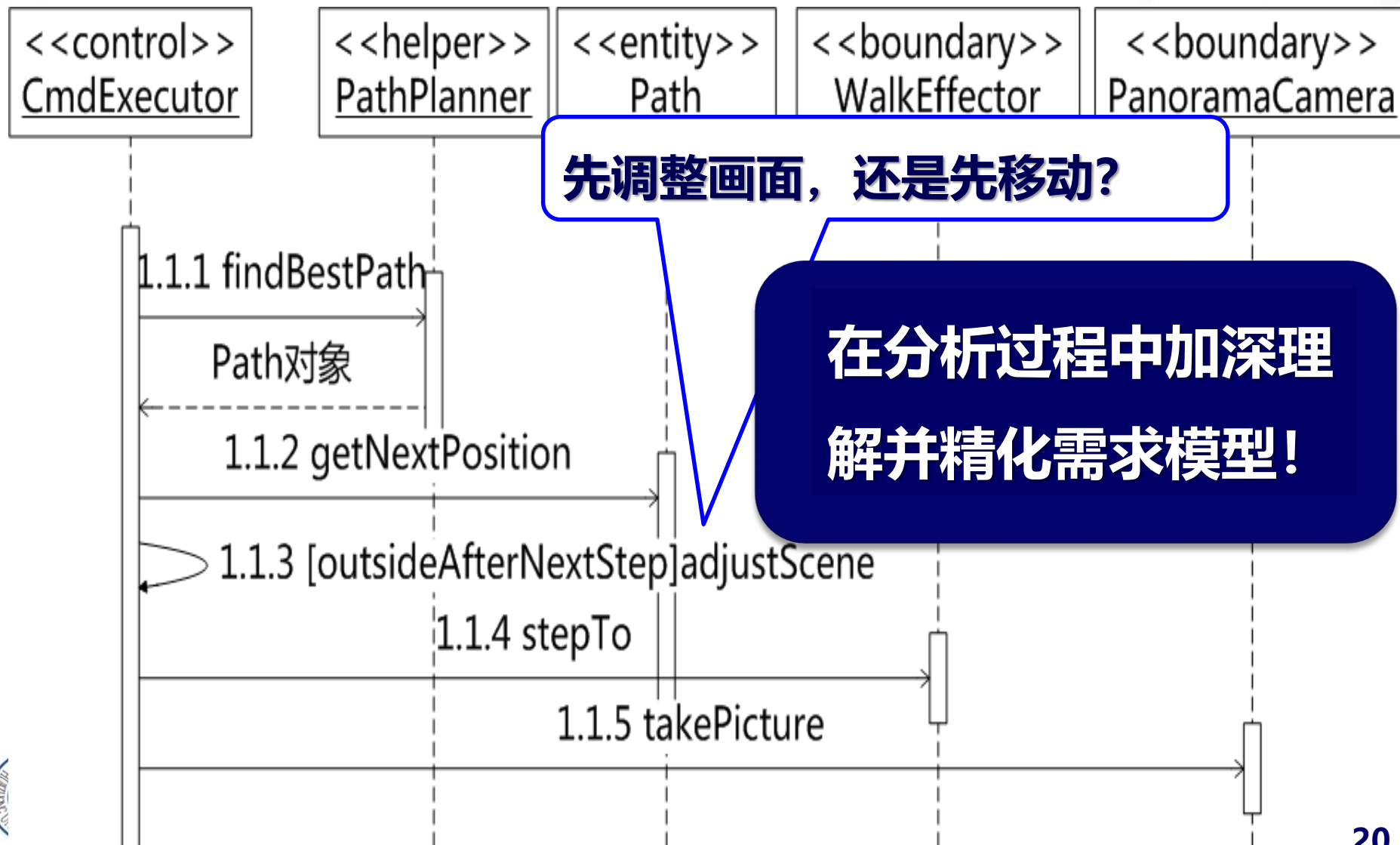
为什么设置辅助类“PathPlanner”？

“移动至指定位置”用例的 基本交互动作序列 (2)

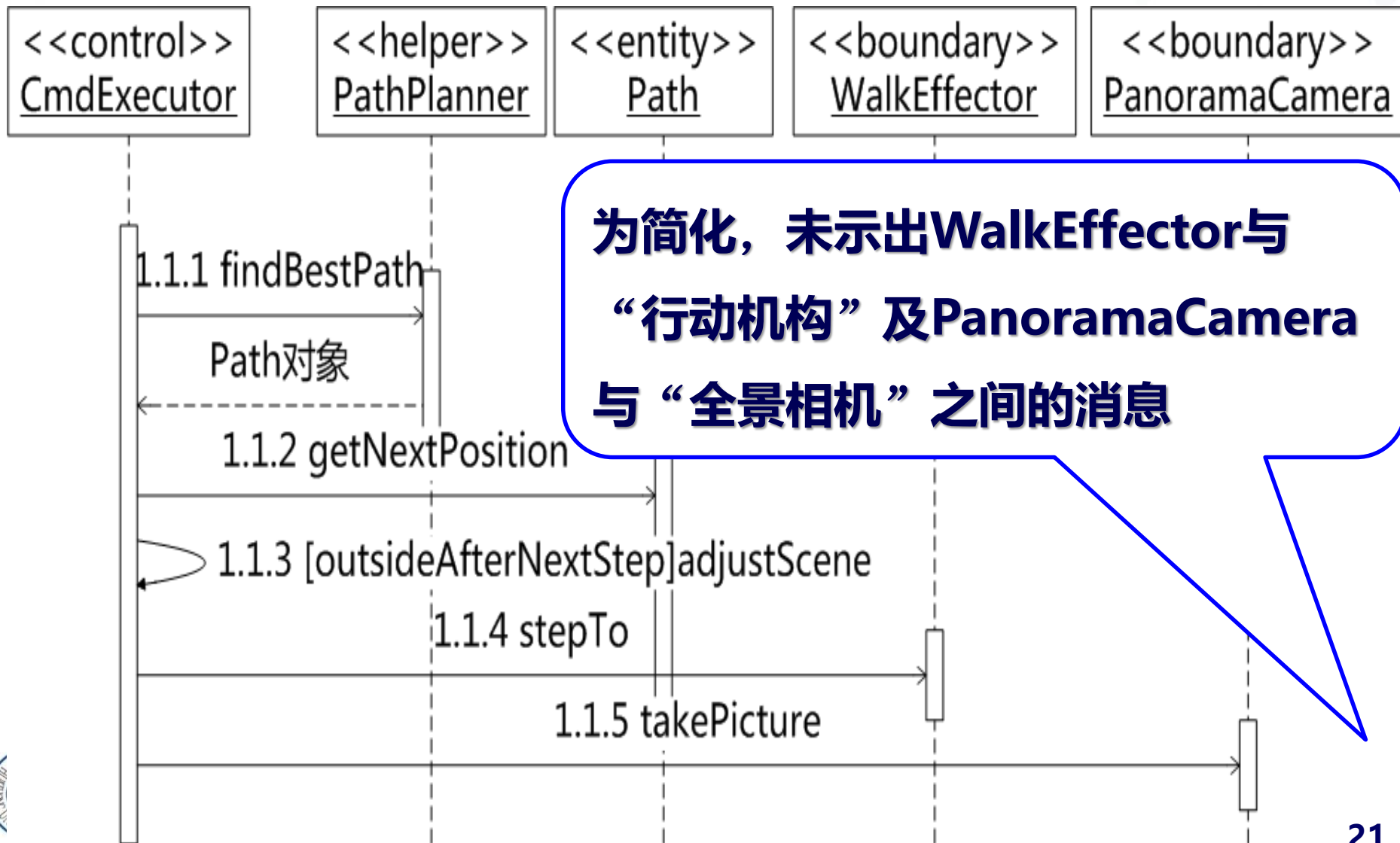
1. 用户指定目的坐标；系统规划从当前位置到达指定位置的最佳路径；
2. 系统发出移动指令；在移动完成后，系统指示全景相机拍照留存，
 - 2.1 系统判断月球车是否已经移出当前画面，如是，移动画面以保证月球车始终位于当前画面之内；
 - 2.2 根据规划路径发出下一步移动指令，如此重复直至到达路径终点。



“移动”用例的顺序图表示（第2步）

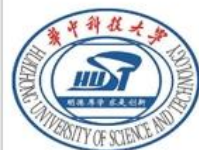


“移动”用例的顺序图表示（第2步）

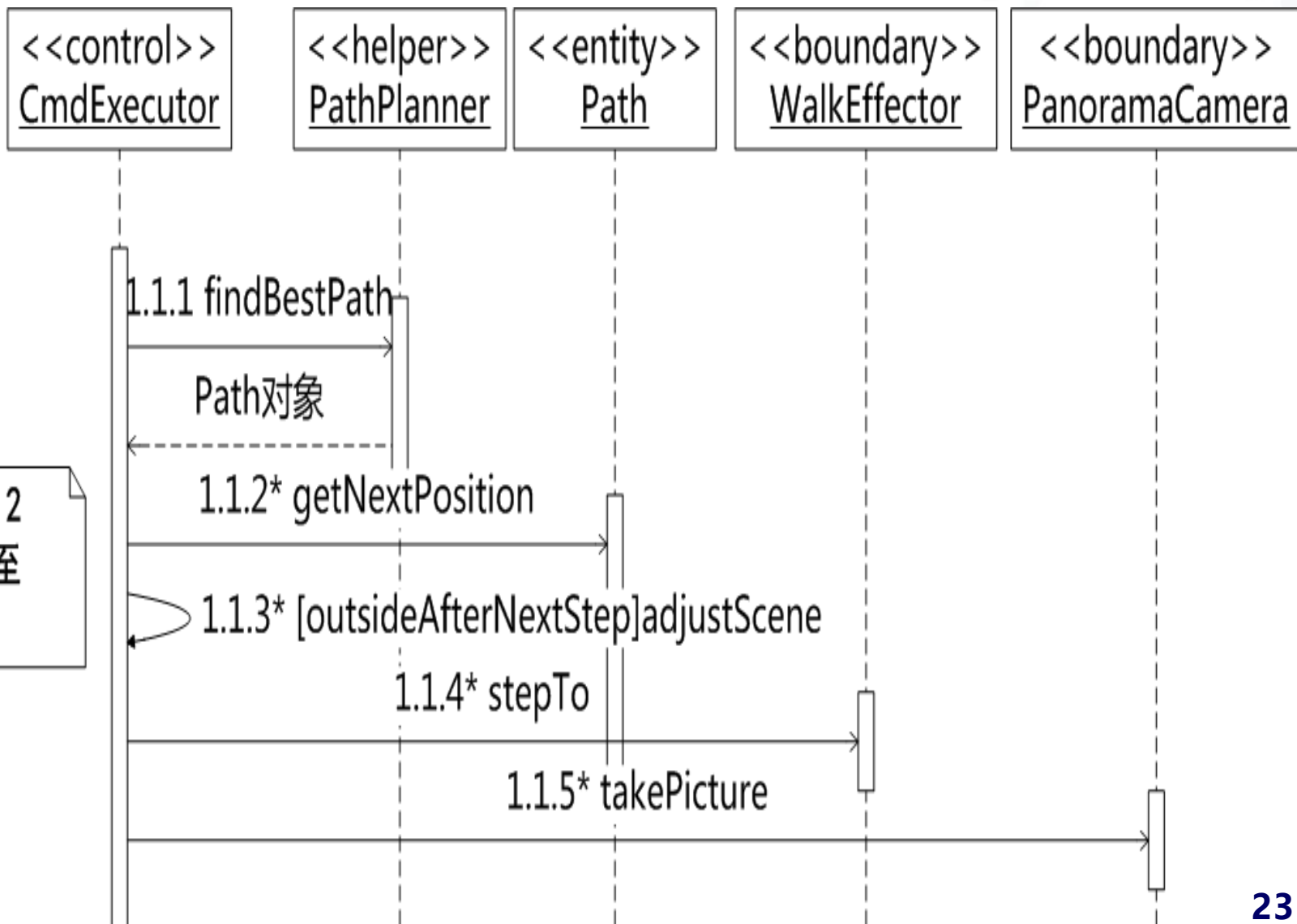


“移动至指定位置”用例的基本交互动作序列 (3)

1. 用户指定目的坐标；系统规划从当前位置到达指定位置的最佳路径；
2. 系统发出移动指令；在移动完成后，系统指示全景相机拍照留存，
 - 2.1 系统判断月球车是否已经移出当前画面，如是，移动画面以保证月球车始终位于当前画面之内；
 - 2.2 根据规划路径发出下一步移动指令，如此重复直至到达路径终点。

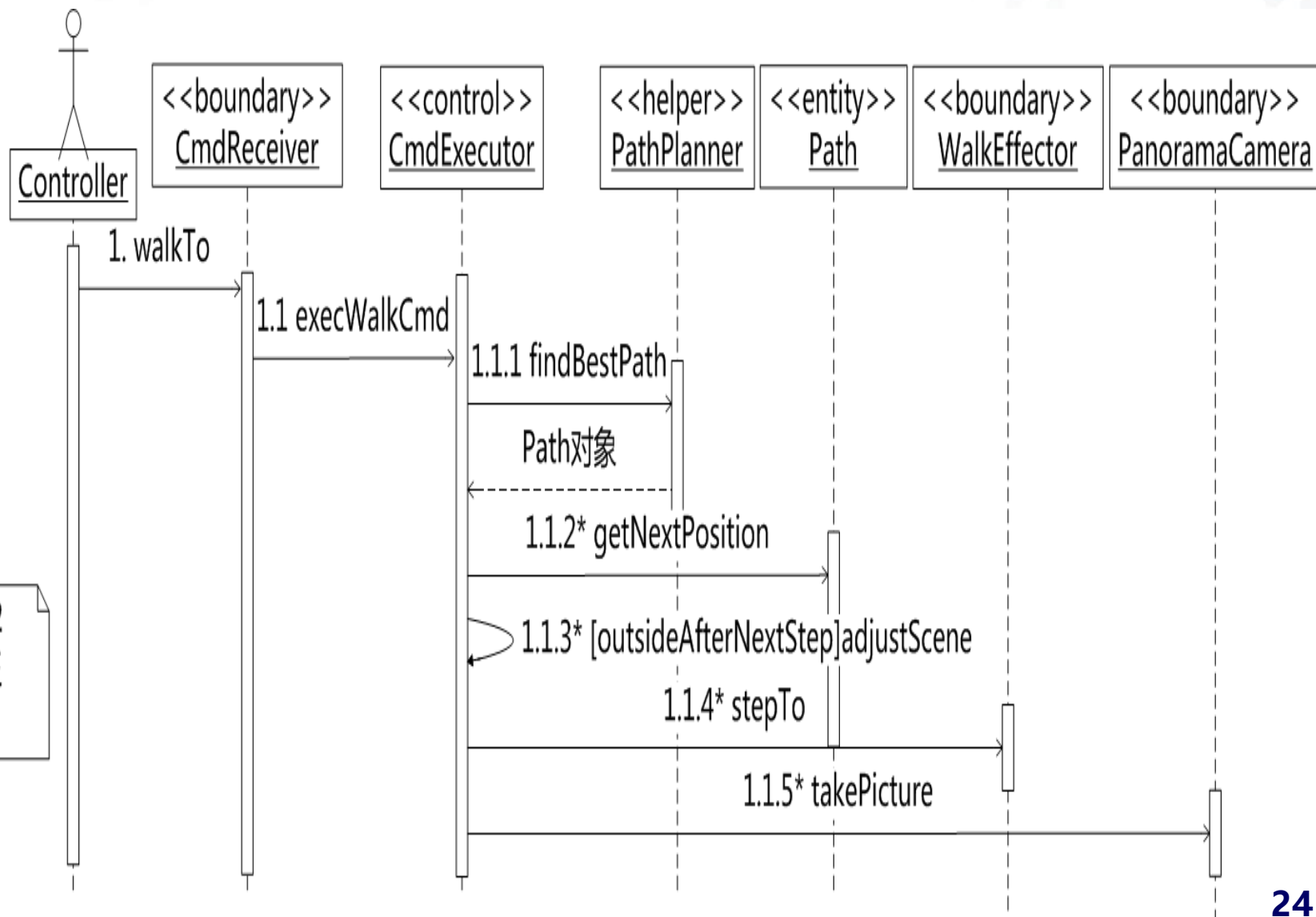


“移动”用例的顺序图表示（第3步）



循环执行1.1.2
至1.1.5, 直至
路径终点

“移动”用例的顺序图表示（“顺序图1”）



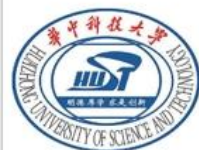
在月球车移动过程中允许暂停

□ 扩展交互动作序列：

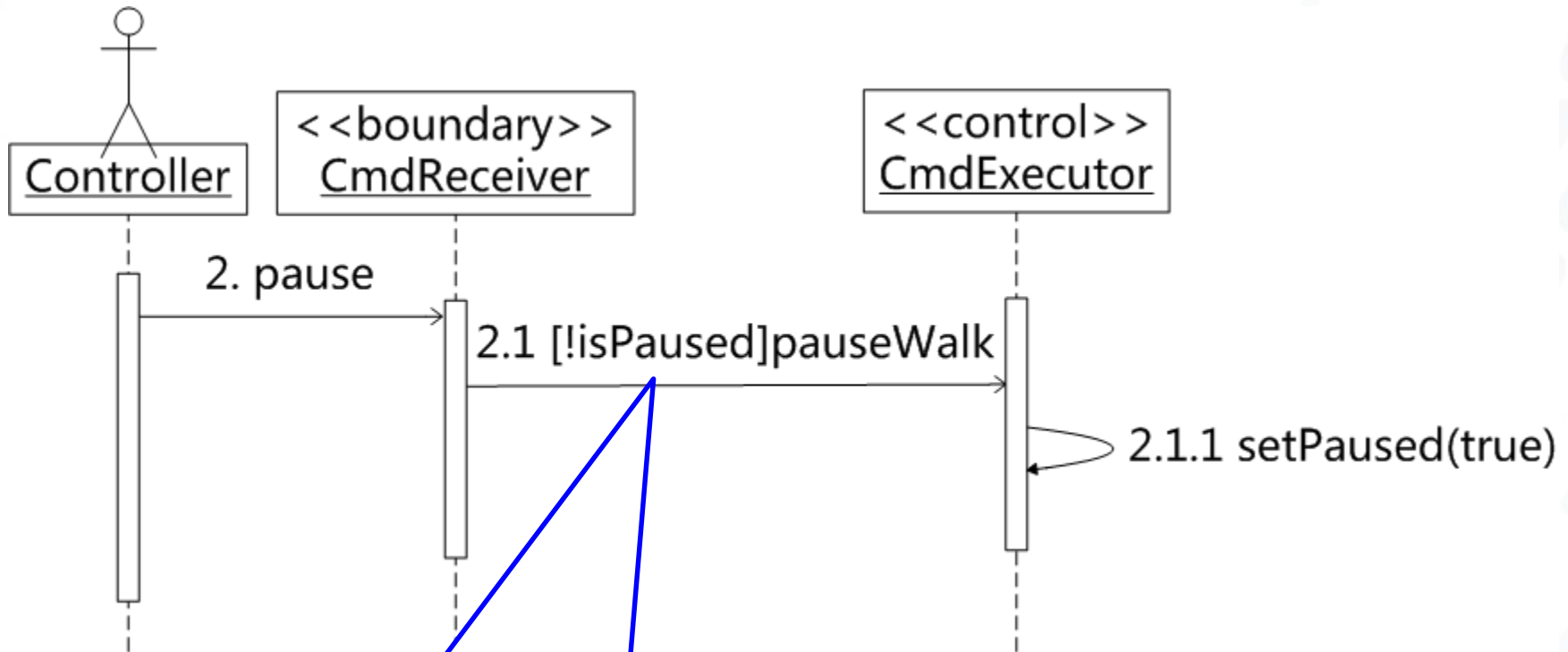
2a. 用户发出暂停指令

2a.1 月球车暂停

2a.2 用户提供新的目的地坐标，系统重新规划路径。在用户发出恢复移动的指令后，返回至2。



允许暂停的顺序图表示（第1步）



为什么设置条件变量“isPaused”？

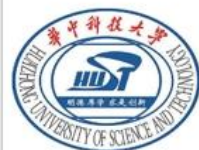
在月球车移动过程中允许暂停 (2)

□ 扩展交互动作序列:

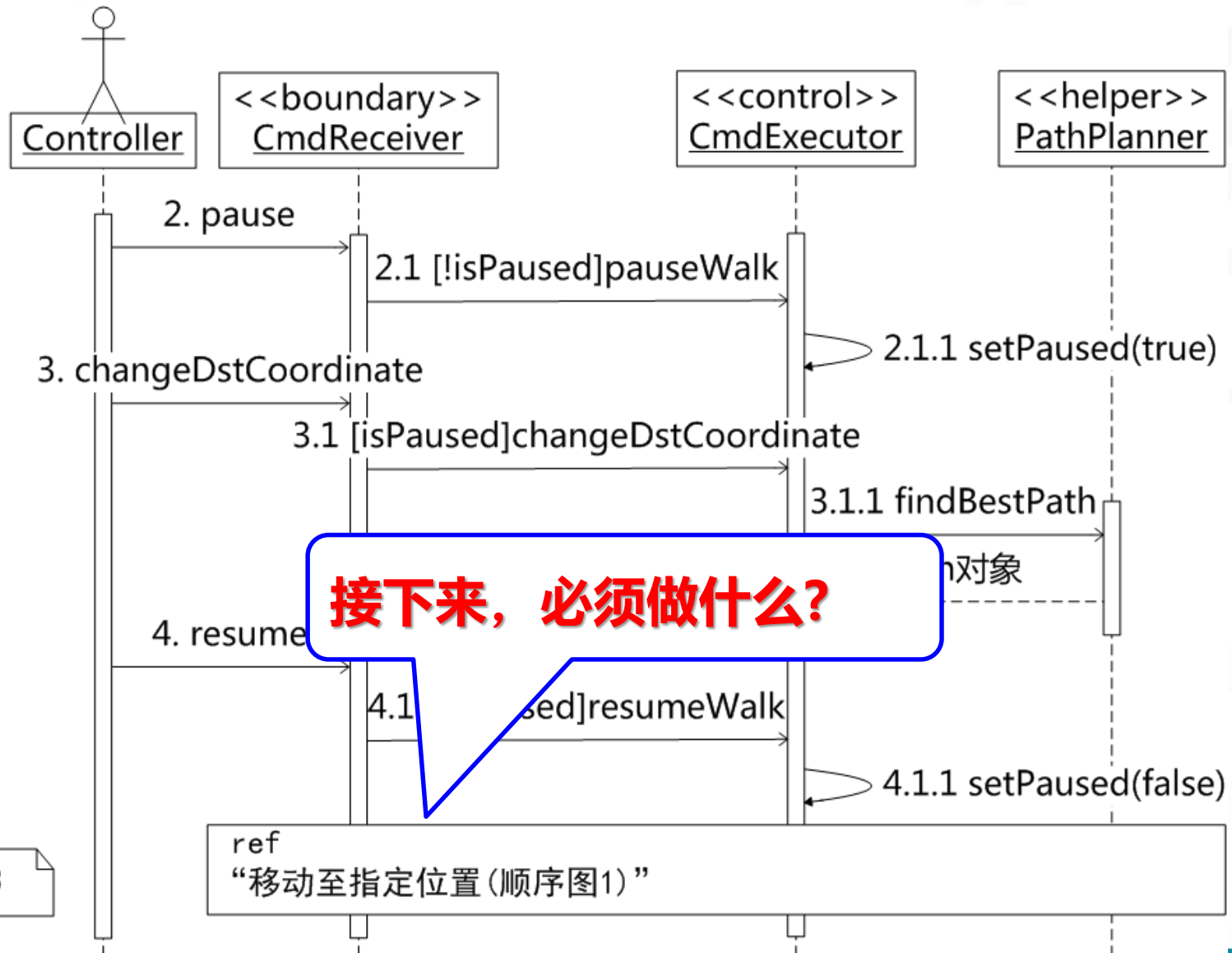
2a. 用户发出暂停指令

2a.1 月球车暂停

2a.2 用户提供新的目的地坐标，系统重新规划路径。在用户发出恢复移动的指令后，返回至2。



允许暂停的顺序图表示 (第2步)

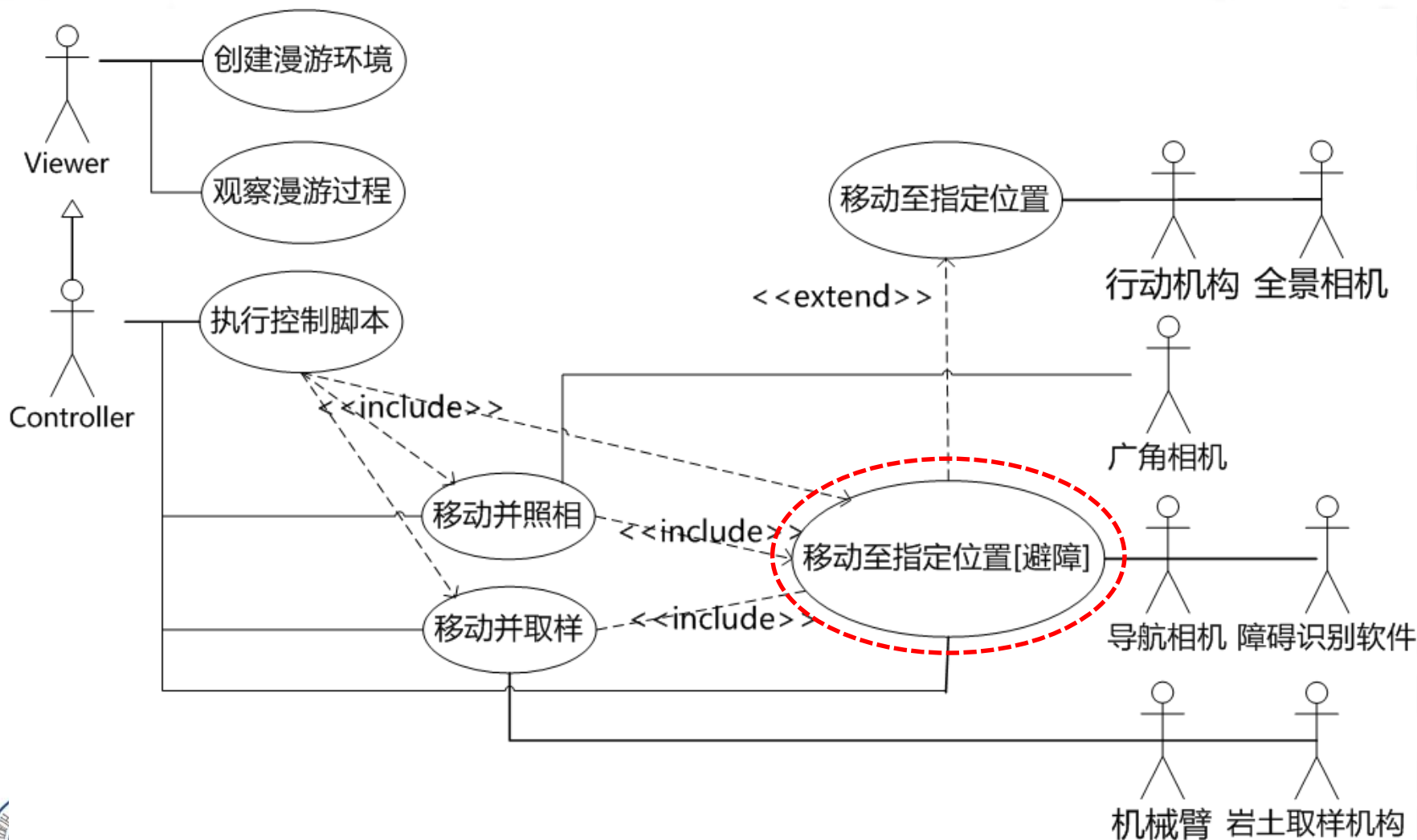


返回至1. 1. 3

必须相应地修改 “移动至指定位置(顺序图1)”

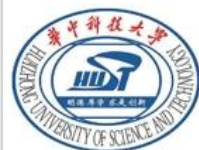


MCS的用例图(2)

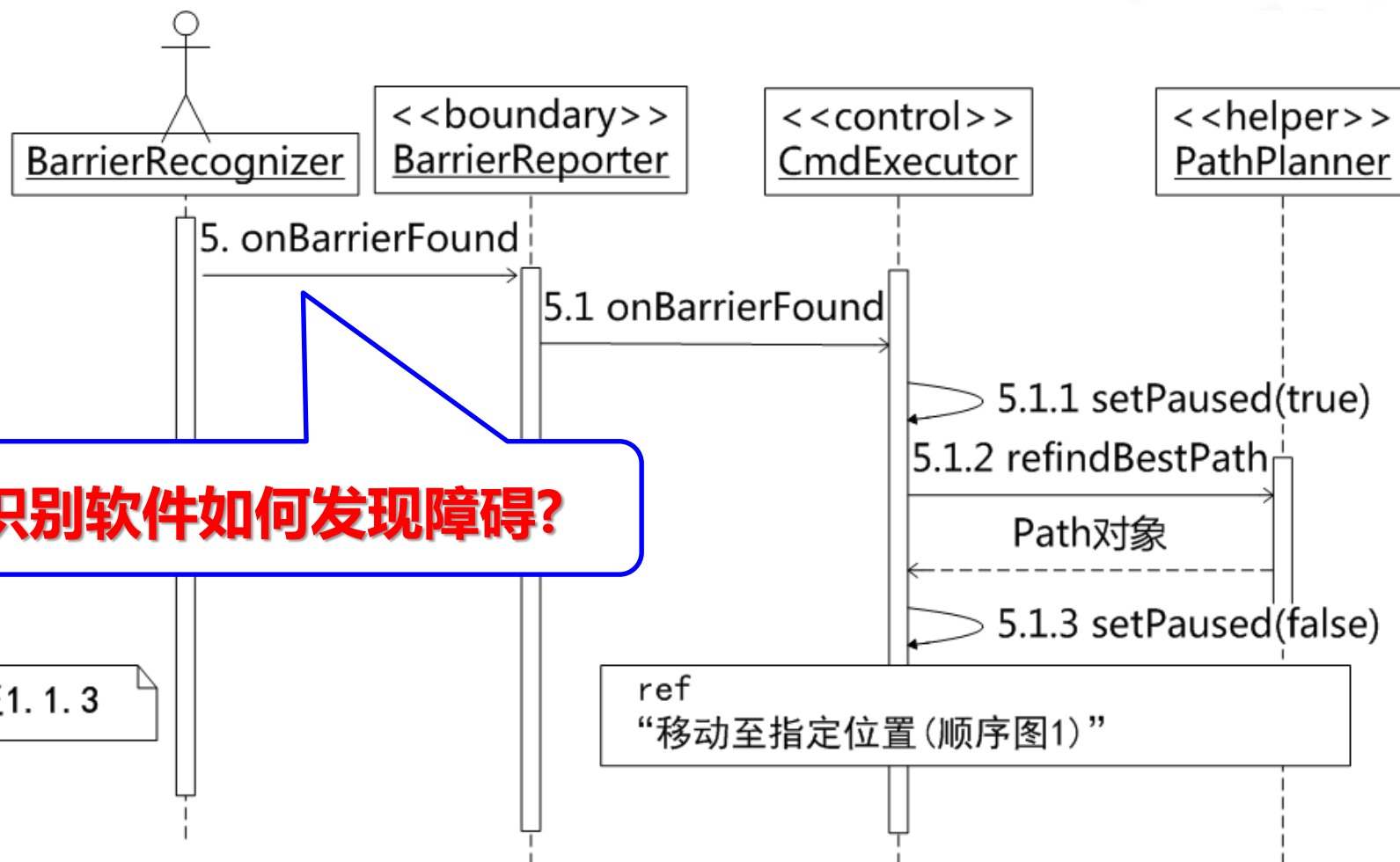


“移动至指定位置[避障]”用例

- 用例名：移动至指定位置[避障]
- 基本交互动作序列：同 移动至指定位置
- 扩展交互动作序列：
 - 2a. 如果导航相机在近前方1米内发现障碍
 - 2a.1 系统重新规划路径，再执行2。



“移动至指定位置[避障]”用例的顺序图表示



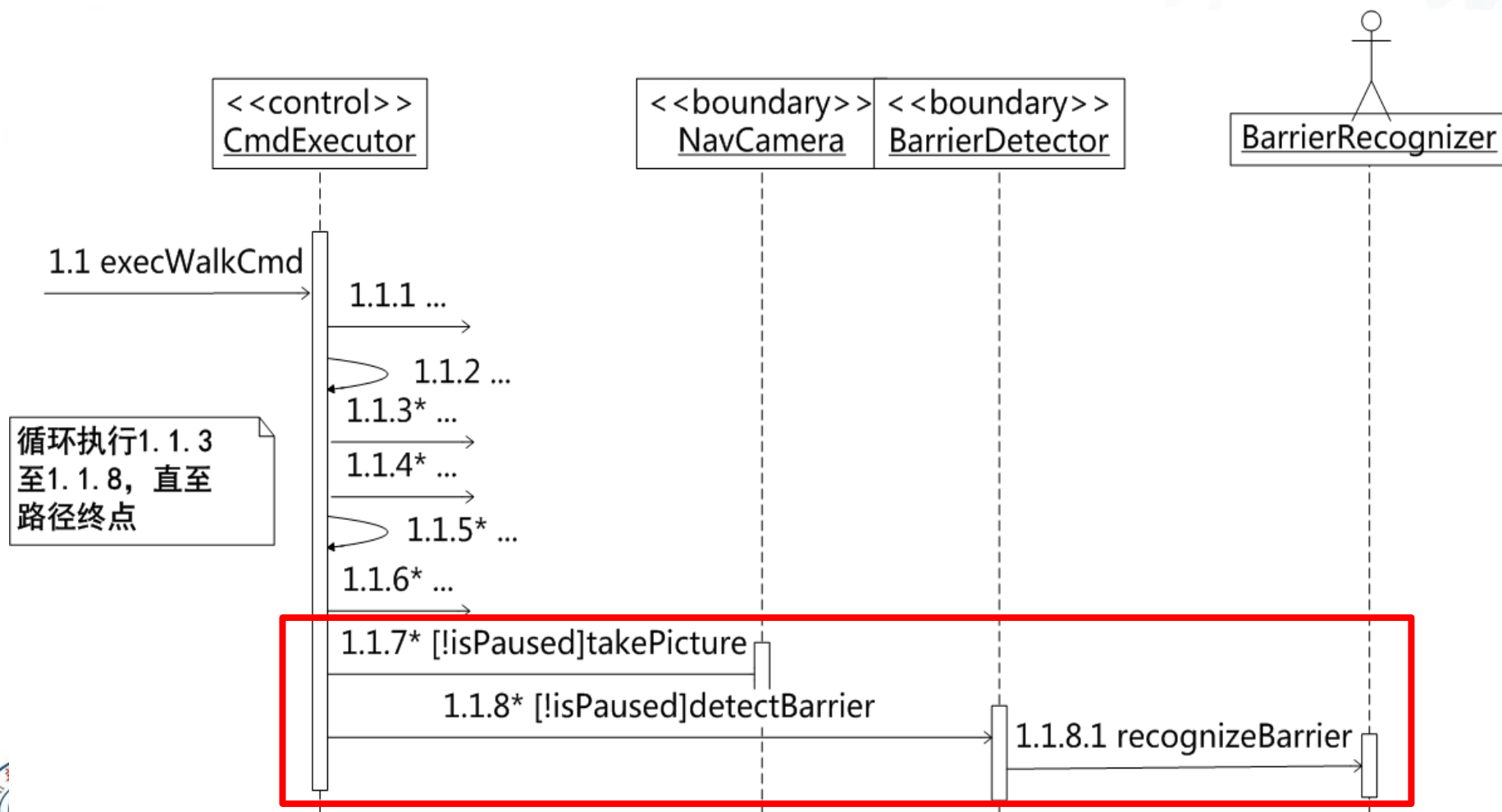
障碍识别软件如何发现障碍？

- 月球车在每移动一步后必须将导航相机拍摄的照片提交给BarrierRecognizer！

因此，必须再次精化“移动至指定位置(顺序图1)”

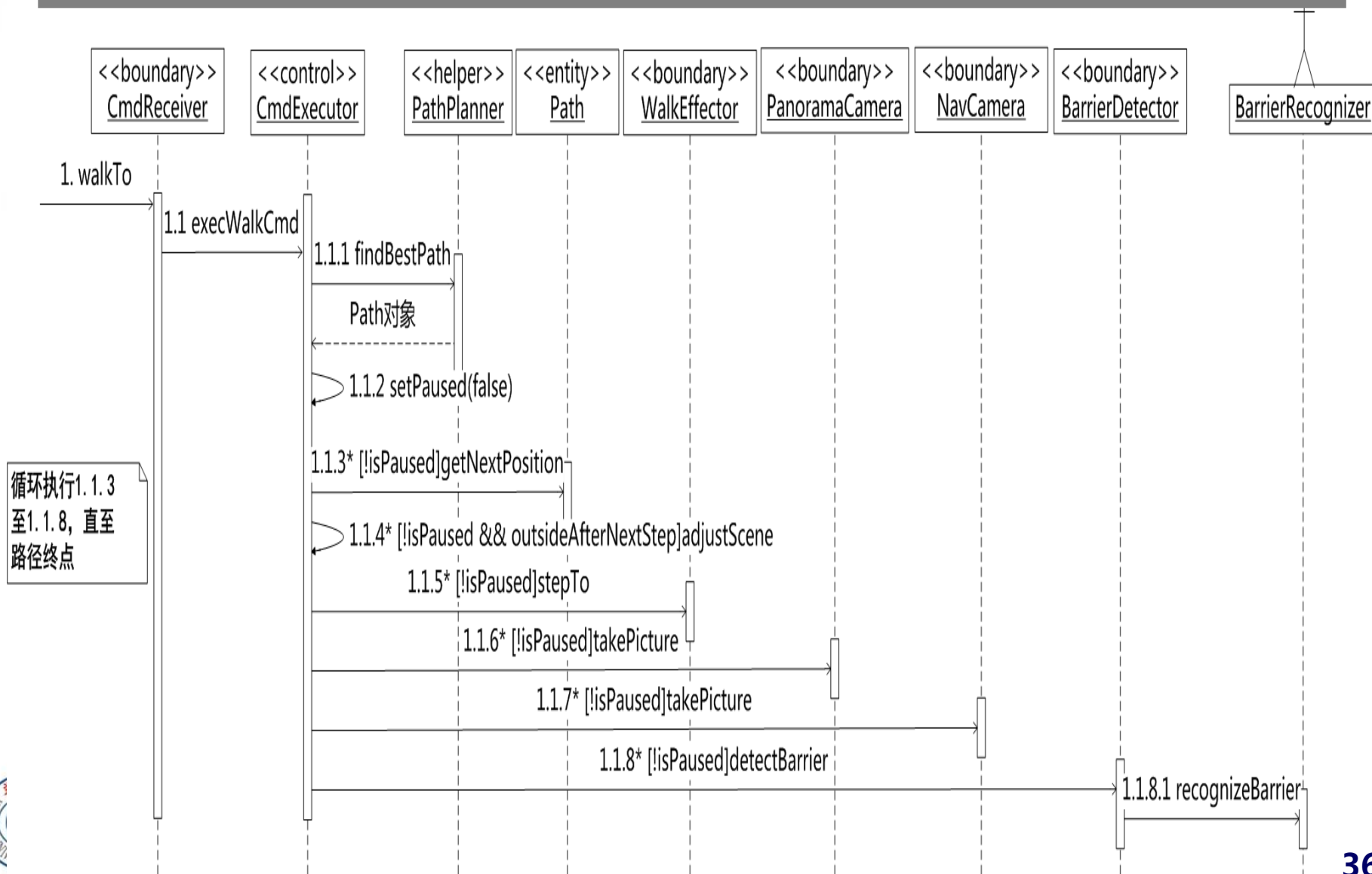
再次精化

“移动至指定位置(顺序图1)”



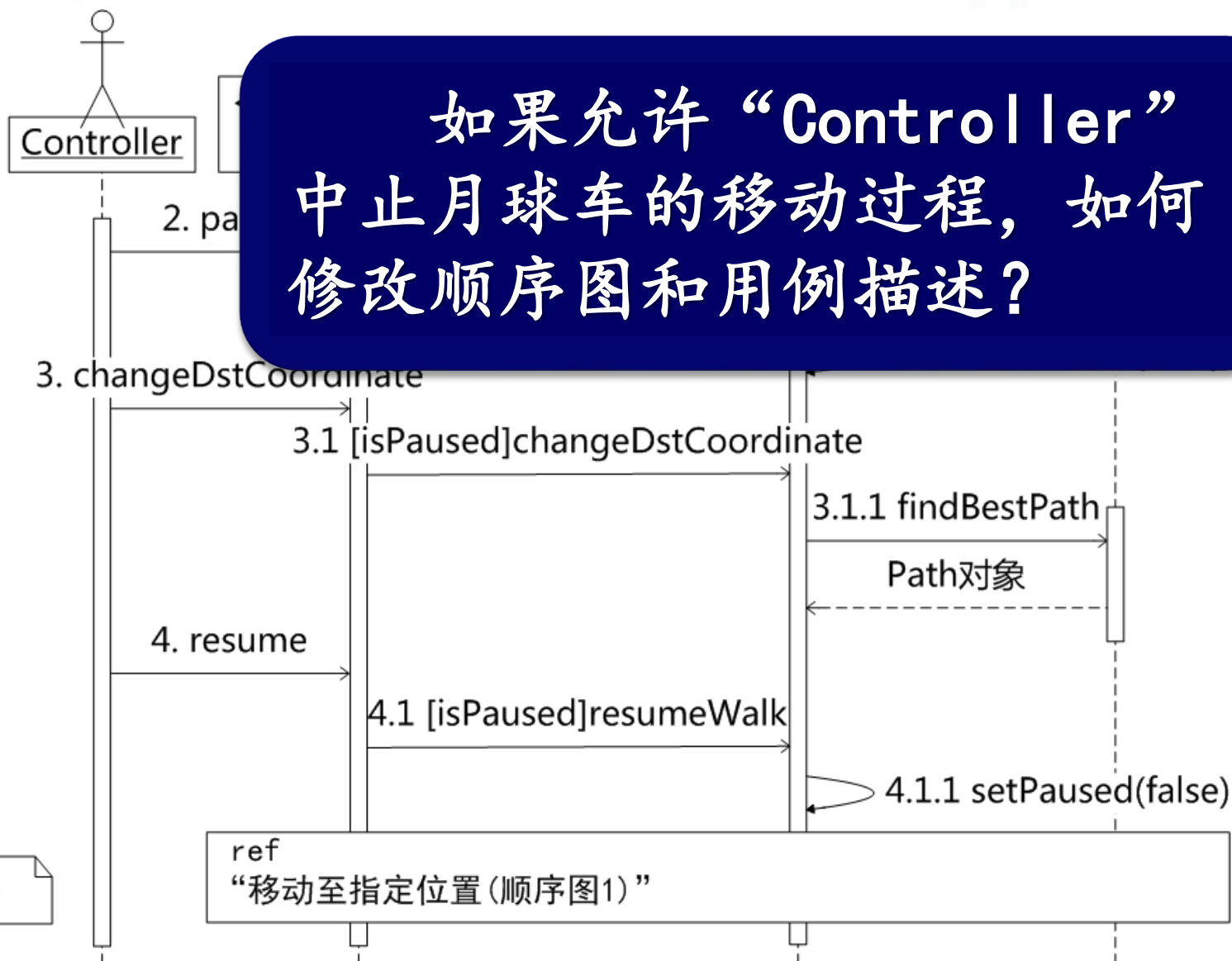
现在，终于完整地构建了
“移动至指定位置”、
“移动至指定位置[避障]”
两个用例的顺序图表示

“移动至指定位置”用例的顺序图1（基本）



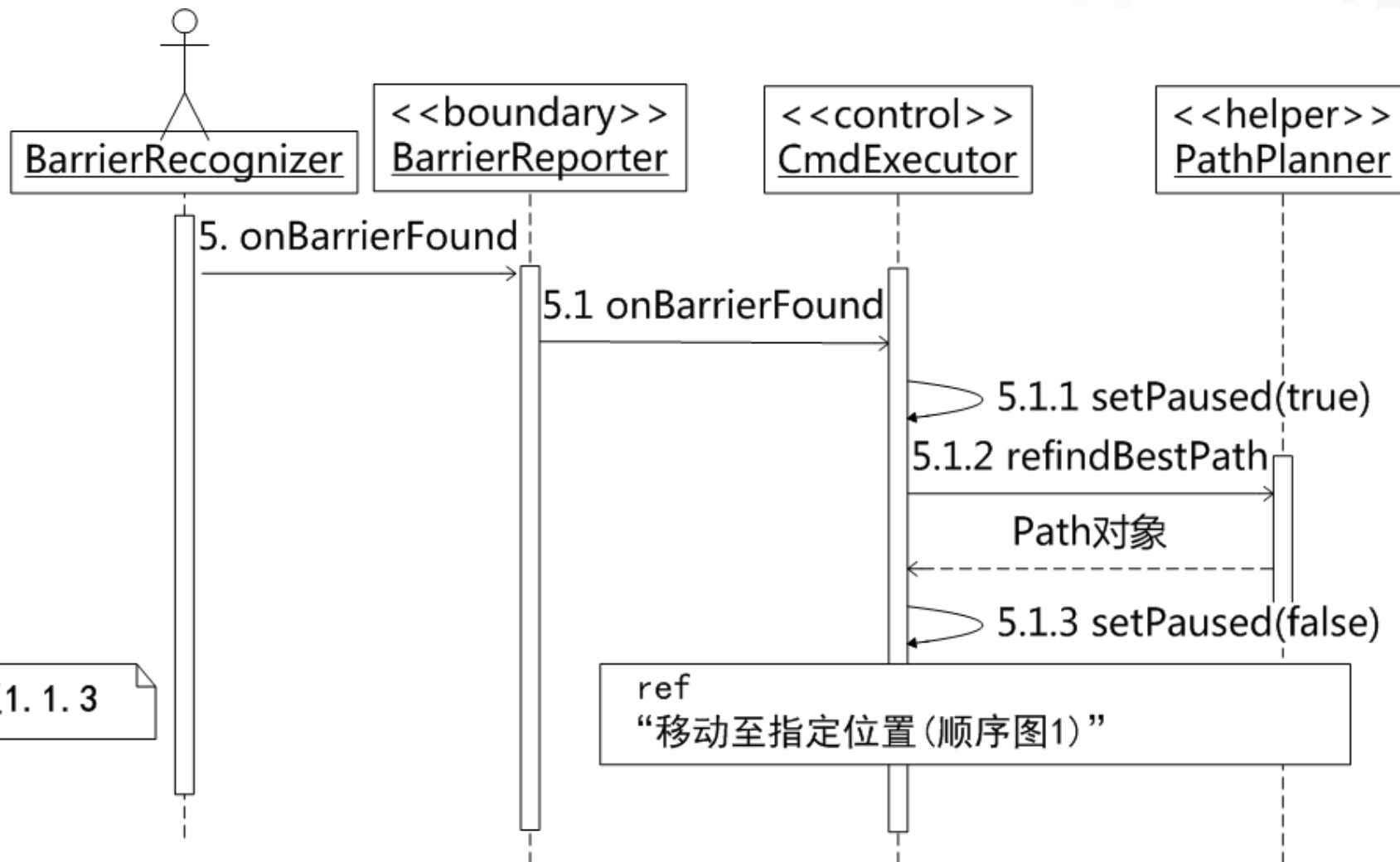
“移动至指定位置”用例的顺序图2（支持暂停）

如果允许“Controller”中止月球车的移动过程，如何修改顺序图和用例描述？



返回至1.1.3

“移动至指定位置[避障]”用例的顺序图



返回至1.1.3

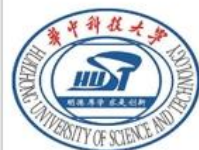
ref
“移动至指定位置 (顺序图1)”

案例分析小结 (1)

□ 以上案例分析生动诠释了：

◆ 在创建分析模型的过程中，不断**加深**对软件需求的**理解**，并且更**全面地理解**软件需求

☑ 例如，“移动至指定位置”用例的基本交互动作必须考虑“暂停”的影响

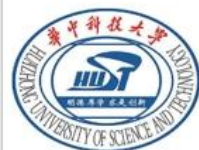


案例分析小结 (2)

□ 以上案例分析生动诠释了：

◆ 通过分析模型中多张相关模型图之间的比对，可以**挖掘**潜在的需求、**查漏补缺**，并提高分析模型图之间**的一致性**

☑ 例如，在每移动一步后必须将导航相机拍摄的照片提交给BarrierRecognizer



案例分析小结 (3)

□ 以上案例分析生动诠释了：

◆ **分析模型（顺序图）比用例模型中的文字描述更加精确，更便于软件设计**

总结：如何展开用例分析？

标识分析类（边界类、控制类、实体类、辅助类）

基于样板 构造用例的顺序图表示

.....

且听下回分解

